Implementazione SoA K-Means con OpenMP

Elia Generali
Università degli Studi di Firenze (UniFi)
elia.generali@edu.unifi.it

Sommario

Questo articolo illustra l'implementazione parallela dell'algoritmo di clustering ***K-Means*** *utilizzando un approccio di layout dati* ***Structure of Arrays (SoA)****. Questa implementazione ha lo scopo di valutare l'efficienza computazionale del metodo SoA sia nel calcolo sequenziale che in quello parallelo, variando dimensioni e numero di cluster e set di dati. Per questo studio è stato utilizzato* ***OpenMP****.*

1 Introduzione

Ad oggi i paradigmi della *machine learning* sono cambiati radicalmente, l'enorme quantità di dati disponibili ha messo in luce il problema dell'aumento del costo computazionale. In un contesto dove la velocità di calcolo e il risparmio di risorse sono diventati tra i più importanti fattori di sviluppo di un qualsiasi algoritmo, diventa essenziale sperimentare metodologie che possano migliorarli in tal senso. Il *clustering* tramite *K-Means* è un metodo ancora molto utilizzato, complice la sua efficacia e semplicità, questo lo rende un metodo particolarmente interessante per valutarne l'efficienza computazionale in diversi contesti.

2 K-Means

Il *clustering K-Means* è un algoritmo di apprendimento non supervisionato con l'obiettivo di partizionare n osservazioni in K cluster, dove ogni osservazione appartiene al cluster con la media (il "centroide") più vicina. L'algoritmo cerca di minimizzare la varianza all'interno di ogni gruppo, ovvero tende a raggruppare i punti di un cluster in modo tale che siano il più possibile vicini tra loro.
A livello formale, il K-Means cerca di minimizzare la *Within-Cluster Sum of Squares (WCSS)*, definita dalla funzione obiettivo J:

$$J = \sum_{i=1}^{K} \sum_{x \in C_i} ||x - \mu_i||^2$$

Dove:

- K è il numero di cluster.
- C_i è l'insieme dei punti nel cluster i.
- μ_i è il centroide del cluster i. $||x-\mu_i||^2$ è la distanza quadratica tra un punto e il suo centroide.

2.1 Algoritmo

L'algoritmo di *clustering K-Means* appena descritto può essere rappresentato sinteticamente dal seguente pseudo-codice:

Algorithm 1: Algoritmo K-Means Parallelo (Ottimizzato SoA)

Input: Dataset D (N punti), Numero di cluster K, Iterazioni massime T
Output: Assegnazione dei punti A, Centroidi finali C

```
Inizializza i centroidi C casualmente dal dataset
for iter ← 1 to T do
    Azzera gli accumulatori globali (somme e conteggi)
    inizio Regione Parallela
        Inizializza accumulatori locali for each thread
        per ogni (in parallelo) punto i ← 1 to N 1. Trova il centroide più vicino calcolando la distanza euclidea (ottimizzata SIMD)
        2. Assegna il punto i al cluster j più vicino
        3. Accumula le coordinate del punto nell'archivio locale del thread
        4. Incrementa il contatore locale dei punti per il cluster j
        sezione critica  Aggiorna i dati globali sommando i risultati locali di ogni thread
    // Fase di aggiornamento (Sequenziale)
    Calcola la posizione dei nuovi centroidi dividendo le somme per il numero di punti assegnati
```

3 Implementazione

In questa sezione viene illustrata l'implementazione dell'algoritmo di *clustering K-Means* con metodo sequenziale e parallelo utilizzando il *layout SoA*.

3.1 SoA

L'organizzazione dei dati secondo il paradigma *Structure of Arrays (SoA)* prevede che tutti i valori appartenenti a una specifica dimensione siano memorizzati in modo

contiguo in memoria. Questo design si contrappone all'approccio classico *Array of Structures (AoS)*, dove i dati sono raggruppati per punto. Nell'implementazione *SoA*, per accedere a una specifica coordinata di un punto, è necessario calcolare l'indice corretto all'interno di un *array* unidimensionale utilizzando la seguente formula:

$$Index = dim_index \times n_points + point_index$$

Dove:

- `dim_index`: rappresenta l'indice della dimensione (coordinata d).
- `n_points`: è il numero totale di campioni nel *dataset*.
- `point_index`: è l'indice del punto specifico all'interno della dimensione.

Questa strategia di *layout* offre benefici critici per l'efficienza computazionale del *K-Means*:

- **Efficienza della *Cache*:** La *SoA* migliora significativamente l'accesso alla memoria per le operazioni che elaborano i dati per dimensione, riducendo drasticamente i *cache miss*.
- **Vettorializzazione SIMD:** Poiché i valori delle dimensioni tra diversi punti sono contigui, il *layout SoA* facilita l'uso delle istruzioni **SIMD (Single Instruction, Multiple Data)**. Il compilatore può caricare blocchi di coordinate direttamente nei registri vettoriali, processando più elementi simultaneamente senza l'*overhead* di riordinamento dei dati (*shuffling*).
- **Località dei Dati:** La struttura assicura che, durante il calcolo delle distanze, i *thread* accedano a posizioni di memoria contigue, ottimizzando l'uso della gerarchia di memoria moderna.

Anche la struttura dei centroidi (`CentroidsSoA`) segue questo schema, garantendo che le fasi di aggiornamento (*update*) e di recupero durante le iterazioni dell'algoritmo siano altrettanto efficienti dal punto di vista dell'accesso alla memoria.

L'adozione dell'organizzazione della memoria tramite *layout Structure of Arrays (SoA)* è stata determinante per superare i limiti di banda passante della memoria (*Memory Wall*). Organizzando le coordinate in vettori dimensionali indipendenti, abbiamo garantito un accesso alla memoria di tipo *strided-unit*, permettendo al compilatore di generare istruzioni vettoriali efficienti. I risultati sperimentali mostrano che questa scelta permette una scalabilità quasi lineare fino al riempimento dei *core* fisici, mantenendo un'alta efficienza anche in regime di *Hyper-Threading*.

3.2 Calcolo della Distanza (SoA)

Il calcolo della distanza euclidea tra i punti del *dataset* e i centroidi rappresenta la fase computazionalmente più onerosa dell'algoritmo **K-Means**. Nella nostra implementazione, l'adozione del *layout Structure of Arrays (SoA)* permette di ottimizzare i pattern di accesso alla memoria e sfruttare appieno il parallelismo a livello di istruzione.

Nel *layout SoA*, i valori di ogni dimensione sono memorizzati contiguamente per tutti i punti del *dataset*. Per calcolare la distanza tra un punto i e un centroide j, l'accesso ai dati avviene tramite un calcolo di offset lineare:

$$\text{Index}_{point} = d \times N_{points} + i \tag{1}$$

$$\text{Index}_{centroid} = d \times K + j \tag{2}$$

Dove d rappresenta la dimensione corrente, N_{points} il numero totale di punti e K il numero di *cluster*. Questo schema garantisce che, durante l'iterazione sulle dimensioni, i dati siano recuperati in modo sequenziale, massimizzando la *cache locality* spaziale.

3.3 Vettorializzazione SIMD e Riduzione

Per accelerare ulteriormente il calcolo, abbiamo utilizzato la direttiva **OpenMP** `#pragma omp simd`, come illustrato nel seguente frammento di codice:

```
inline float compute_distance(const Dataset& data, size_t point_idx, const Centroids& centroids, size_t centroid_idx) {

  float dist = 0.0f;
  const size_t n_dims = data.n_dims;
  const size_t n_points = data.n_points;
  const size_t k = centroids.k;
  const float* p_ptr = data.data.data();
  const float* c_ptr = centroids.data.data();

  #pragma omp simd reduction(+:dist)
  for (size_t d = 0; d < n_dims; ++d) {
    float diff = p_ptr[d * n_points + point_idx] - c_ptr[d * k + centroid_idx];
    dist += diff * diff;
  }
  return dist;
}
```

Listing 1: Implementazione SoA del calcolo della distanza euclidea con ottimizzazione SIMD.

L'integrazione di questa logica offre tre vantaggi fondamentali:

- ***Vectorization***: Il compilatore trasforma le operazioni del ciclo in istruzioni SIMD (*Single Instruction, Multiple Data*), processando più dimensioni simultaneamente.
- ***Reduction Process***: La clausola `reduction` permette a ogni *thread* di mantenere una copia privata della variabile `dist`, combinando i risultati parziali alla fine del ciclo per garantire la *thread-safety* senza *overhead* di sincronizzazione esplicita.
- **Efficienza della Cache**: Poiché i dati sono allineati per dimensione, si riducono drasticamente i *cache miss* rispetto a un *layout* di tipo AoS, dove l'accesso sarebbe di tipo *strided*.

3.3.1 Ottimizzazione del Calcolo

Un'ottimizzazione critica introdotta nella funzione di calcolo della distanza riguarda la gestione delle operazioni aritmetiche elementari. Invece di utilizzare la funzione generica `std::pow(diff, 2)` fornita dalla libreria standard, l'implementazione adotta la **moltiplicazione diretta** (`diff * diff`).

Questa scelta è motivata dal fatto che `std::pow` è progettata per gestire esponenti frazionari e casi complessi, introducendo un *overhead* di chiamate a funzione e logiche di controllo non necessarie per un elevamento al quadrato. La moltiplicazione diretta, invece, viene tradotta dal compilatore in una singola istruzione macchina ad alta velocità, riducendo drasticamente i cicli di *clock* necessari per ogni dimensione processata.

L'efficacia di questa ottimizzazione è massimizzata dal *layout SoA*, poiché la contiguità dei dati permette alla CPU di mantenere un flusso costante di operandi verso l'unità aritmetico-logica (ALU), evitando colli di bottiglia legati alla latenza delle funzioni matematiche esterne.

4 K-Means Sequenziale

La prima implementazione dell'algoritmo K-Means tramite *SoA* utilizza una strategia sequenziale.
L'implementazione segue il processo dell'algoritmo con una serie di strategie per aumentarne la velocità di convergenza:

- **Assegnazione e Ottimizzazione SIMD:** Il cuore computazionale risiede nel calcolo della distanza Euclidea tra N punti e K centroidi. Attraverso la Vettorizzazione via OpenMP con la direttiva `#pragma omp simd` istruisce il compilatore a trasformare il ciclo di calcolo in istruzioni vettoriali. Invece di calcolare una distanza scalare alla volta, il processore esegue operazioni su vettori di dati, processando più dimensioni o più punti contemporaneamente in un singolo ciclo di *clock*. Questo riduce drasticamente il numero di istruzioni totali e massimizza il *throughput* della CPU.
- **Aggiornamento, Accumulo e Riduzione:** Per aggiornare la posizione dei centroidi, l'algoritmo deve calcolare il valore atteso (media) delle posizioni dei punti assegnati a ciascun cluster. Attraverso un ***Buffer* Temporaneo di Accumulo** si evitano accessi multipli e disordinati ai centroidi, i nuovi valori vengono accumulati in uno spazio di memoria intermedio. Inoltre vengono inizializzati dei **Contatori di *Cluster***, viene mantenuto un array di interi di dimensione K che memorizza la cardinalità di ogni cluster $|C_j|$. Questo è necessario perché l'algoritmo è sequenziale e deve ricostruire il denominatore per la media finale.
- **Normalizzazione e Calcolo dei Nuovi Centroidi:** Questo è il ciclo finale sui K centroidi dove ogni coordinata accumulata nel buffer viene divisa per il rispettivo contatore di cluster. L'output di questa fase diventa l'input (i centroidi aggiornati) per l'iterazione successiva, fino al raggiungimento di MAX_ITER

In sintesi, questo sistema di calcolo sequenziale cerca di massimizzare l'efficienza computazionale del singolo core della CPU. Invece di elaborare un dato alla volta, il sistema cerca di "parallelizzare" il calcolo all'interno del singolo processore, usando le istruzioni SIMD (Single Instruction, Multiple Data), si caricano interi vettori di dati nei registri della CPU permettendo di sfruttare ogni ciclo di *clock* disponibile.

5 K-Means Parallelo

In questa seconda parte viene integrata la logica di calcolo parallelo (*multithread*). Questa strategia può essere riassunta nei punti seguenti:

1. **Modello di Condivisione (Orchestrazione):** Si definisce un confine netto tra dati Globali (Shared), come i punti e i centroidi in lettura, e dati Locali (Private). L'uso di *firstprivate* garantisce che ogni thread lavori su registri propri per le variabili di controllo, eliminando la latenza di accesso alla memoria condivisa.
2. **Accumulo Locale (Anti-Contention):** Ogni thread gestisce una copia privata degli accumulatori dei centroidi. Questa tecnica elimina il False Sharing (conflitti tra cache line) e permette a ogni core di processare i propri dati alla massima velocità senza dover bloccare il bus di sistema per scrivere i risultati parziali.
3. **Scheduling Statico (Località):** Il carico viene suddiviso in blocchi fissi (static). Essendo il K-means computazionalmente uniforme, lo scheduling statico è il più efficiente perché minimizza l'overhead di gestione del runtime e massimizza il riutilizzo dei dati nelle cache L1/L2.
4. **Ottimizzazione SoA vs AoS (Vettorizzazione):** Il sistema adatta l'accesso alla memoria al layout dei dati. Il layout SoA (Structure of Arrays), in particolare, è sfruttato per permettere un caricamento lineare nei registri SIMD, garantendo

che ogni istruzione del processore operi su un vettore completo di coordinate anziché su una singola variabile.

5. **Sincronizzazione Differita (Reduction):** La sincronizzazione globale è ridotta al minimo indispensabile. Invece di sincronizzare ogni singolo punto, si esegue un unico merge finale dei risultati locali tramite una critical section per ogni iterazione. Questo sposta il collo di bottiglia dalla comunicazione tra core al calcolo puro.

6. **Normalizzazione Sequenziale:** Il ricalcolo finale della posizione dei centroidi (la media) non viene parallelizzato. Data l'irrilevanza del suo costo computazionale rispetto alla fase di assegnazione, si evita l'overhead di creazione/gestione dei thread per un'operazione che richiederebbe solo pochi cicli di clock.

6 Risultati

Di seguito vengono riportati i risultati dell'implementazione *SoA* per l'algoritmo *K-Means*, testato secondo diversi parametri.
L'unità *hardware* utilizzata è una CPU ***AMD Ryzen 5 4600H*** fornita di 6 *core* fisici, 12 *thread* logici e 3.00 GHz di velocità di base.

6.1 *Speedup* - Punti del *dataset*

Tabella 1: Speedup for SoA implementation with varying number of points and threads.

	Threads				
Points	1	2	4	8	16
10000	0.8	1.333	1.333	1.333	1.333
100000	0.953	1.708	2.733	3.417	3.417
1000000	0.959	1.919	3.132	4.218	4.176
10000000	0.96	1.87	2.851	4.159	4.512

L'analisi dei risultati ottenuti evidenzia una correlazione diretta tra la dimensione del dataset e l'efficacia del parallelismo.
Il comportamento dell'algoritmo cambia drasticamente con l'aumentare del *dataset*. Analizzando la configurazione con 10^7 punti, possiamo trarre le seguenti conclusioni basate sull'architettura *hardware* utilizzata (6 core fisici / 12 core logici):

- **Scalabilità nei *Core* Fisici:** Fino a 4 *thread*, l'incremento delle prestazioni è quasi lineare. Il sistema sfrutta appieno i *core* indipendenti e le rispettive *cache* L1 e L2, riducendo i tempi di esecuzione in modo proporzionale alle risorse impiegate.

- **Saturazione e *Hyper-Threading*:** Il passaggio da 8 a 16 *thread* mostra un incremento marginale dello *speedup* (da 4.159 a 4.512), evidenziando *plateau* prestazionale. Avendo a disposizione solo 6 *core* fisici, l'esecuzione di 16 *thread* forza il sistema a fare affidamento massiccio sull'*Hyper-Threading*. In un algoritmo come il *K-Means*, caratterizzato da un elevato numero di operazioni in virgola mobile e frequenti accessi in memoria, i *thread* logici finiscono per competere per le stesse unità funzionali (ALU/FPU) e per la stessa ampiezza di banda del *bus* di memoria, limitando il guadagno prestazionale.

- **Efficienza Parallela:** L'efficienza massima si registra nel *range* 2-6 *thread*. Oltre questa soglia, l'efficienza parallela decresce a causa della contesa delle risorse (*memory contention*) e del peso crescente delle sezioni critiche o di riduzione (fase di aggiornamento dei centroidi), che rappresentano la quota non parallelizzabile del codice secondo la *Legge di Amdahl*.

6.2 *Speedup* - Efficienza

Un'altra analisi interessante riguarda come varia l'**efficienza** parallela rispetto l'utilizzo dei *threads*, formalizzata come:

$$E = \frac{Speedup}{Threads} \times 100$$

Tabella 2: Analisi dello Speedup e dell'Efficienza Parallela (10^7 punti)

Threads	**1**	**2**	**4**	**8**	**16**
Speedup	0.960	1.870	2.851	4.159	4.512
Efficienza	96.0%	93.5%	71.3%	52.0%	28.2%

La metrica dell'efficienza (E) fornisce una comprensione più profonda dell'utilizzo delle risorse:

- **Alta Efficienza ($N \leq 4$):** L'implementazione mantiene un'efficienza superiore al 70%, dimostrando che il *layout SoA* riduce con successo i *cache miss* e favorisce una vettorizzazione efficace dei dati.

- **Crollo dell'Efficienza ($N > 8$):** L'efficienza cala drasticamente fino al 28.2% con 16 *thread*. Questo declino è dovuto ad una *oversubscription hardware*. In un *task* intensivo come il *K-Means*, il collo di bottiglia si sposta dal puro calcolo alla larghezza di banda della memoria. Quando 16 *thread* logici tentano di accedere simultaneamente al *bus* di memoria su soli 6 *core* fisici, la contesa dei dati e il fenomeno del *cache thrashing* portano al *plateau* osservato.

6.3 *Speedup* - Dimensionalità

La Tabella 3 illustra le prestazioni dell'implementazione *SoA* mantenendo fisso il numero di punti (10^6) e il numero di *cluster* ($K = 16$), variando la dimensionalità dei dati da 2 a 128 dimensioni. Questa analisi è fondamentale per comprendere come l'algoritmo gestisca l'aumento del carico computazionale per singola

Tabella 3: Speedup for SoA implementation with varying dimensionality (Fixed: 10^6 points, $K = 16$).

	Threads				
Dim	1	2	4	8	16
2D	0.963	1.912	3.541	3.639	3.853
4D	0.938	1.823	2.939	3.789	3.6
8D	0.956	1.728	3.037	3.896	4.13
16D	0.948	1.854	3	3.902	4.327
32D	0.995	1.961	2.964	3.796	4.249
64D	1.02	2.04	3.361	4.362	4.465
128D	1.017	1.974	3.286	4.448	4.776

iterazione.
Si osserva un trend generale positivo, lo *speedup* tende a migliorare leggermente all'aumentare della dimensionalità, specialmente nelle configurazioni a 8 e 16 *thread*.

Con dimensionalità elevata (128D), si ottiene lo *speedup* massimo di 4.776 a 16 *thread*. Questo accade perché, all'aumentare delle dimensioni, il calcolo della distanza euclidea diventa molto più oneroso rispetto all'*overhead* di gestione dei *thread*. Aumenta il rapporto tra calcolo e comunicazione, permettendo ai *thread* di lavorare più a lungo prima di dover sincronizzare i dati.

Con dimensionalità ridotta (2D), lo *speedup* a 16 *thread* è più limitato (3.853). Qui il calcolo è così rapido che l'accesso alla memoria e la sincronizzazione pesano proporzionalmente di più sul tempo totale.

Si osserva un risultato significativo nelle configurazioni 64D e 128D con 1 o 2 *thread*, dove lo *speedup* è leggermente superiore al valore ideale (es. 1.017 e 1.02).

6.4 *Speedup - K-cluster*

Tabella 4: Speedup for SoA implementation with varying number of clusters (Fixed: 10^6 points, 8D).

	Threads				
K	1	2	4	8	16
2	0.848	1.527	2.625	3.5	3.5
4	0.909	1.733	2.407	3.939	3.939
8	0.918	1.798	3.328	4.055	4.288
16	0.97	1.905	3.289	4.626	4.73
32	0.993	1.935	3.668	4.665	4.981
64	1.001	1.976	3.692	4.383	4.888
128	1.019	1.987	3.816	4.677	5.259

La Tabella 4 presenta lo *speedup* dell'implementazione *SoA* mantenendo fissi il numero di punti (10^6) e la dimensionalità (8D) ma variando il numero di cluster K da 2 a 128. Questa analisi permette di valutare la scalabilità dell'algoritmo rispetto alla complessità del problema di partizionamento.

Si osserva un miglioramento netto e costante dello *speedup* all'aumentare di K:

- $K = 2$**:** Con solo due cluster, lo *speedup* a 16 *thread* è molto limitato (3.5). In questo scenario, il calcolo della distanza euclidea per ogni punto è estremamente rapido. Di conseguenza, il tempo speso per la sincronizzazione dei *thread* e per la riduzione finale dei centroidi diventa predominante rispetto al calcolo effettivo.

- $K = 128$**:** All'aumentare dei cluster, lo *speedup* a 16 *thread* raggiunge il valore massimo registrato in tutti i test (5.259). Questo accade perché il carico di lavoro computazionale per ogni punto aumenta linearmente con K (ogni punto deve essere confrontato con 128 centroidi invece di 2). L'algoritmo diventa quindi *compute-bound* (limitato dal calcolo) piuttosto che *memory-bound* o limitato dall'*overhead* di sincronizzazione, permettendo ai 16 *thread* logici di lavorare con un'efficienza superiore.

Un aspetto tecnico critico del *K-means* parallelo è la fase di aggiornamento dei centroidi, in cui i risultati parziali di ogni *thread* devono essere sommati.
Con K basso, la riduzione è veloce ma lo *speedup* è basso perché c'è poco lavoro parallelo.
Con K alto, sebbene la riduzione sia più onerosa (bisogna sommare 128 vettori), l'abbondanza di calcolo nella fase di assegnazione dei punti "nasconde" efficacemente questo costo, portando a una scalabilità globale migliore. Si osserva uno *speedup* quasi ideale a bassi *thread* per $K = 128$, i valori di *speedup* a 1 e 2 *thread* (1.019 e 1.987) rasentano la perfezione teorica.

Questo conferma che l'approccio *SoA* massimizza l'utilizzo delle *cache* quando il processore deve iterare ripetutamente sui centroidi. La struttura dei dati permette di mantenere i vettori dei centroidi più vicini possibile alle unità di calcolo, riducendo i tempi di latenza.

6.5 *Speedup* - Risultati finali

Tabella 5: Speedup della versione SoA per il dataset massimo (10^7 punti)

	Threads							
M	1	2	4	8	16	24	32	48
SoA	0.96	1.87	2.851	4.159	4.512	4.833	4.896	4.937

La Tabella 5 analizza lo *speedup* della versione *SoA* su un *dataset* di 10^7 punti, estendendo il numero di *thread* fino a 48. Questa analisi è fondamentale per identificare il punto di saturazione definitiva delle risorse *hardware*.

Si osserva che lo *speedup* entra in una fase di *plateau* quasi totale dopo i 16 *thread*. Nonostante l'aumento massiccio dei *thread* (da 16 a 48), lo *speedup* guadagna

solo pochi decimali, passando da 4.512 a 4.937. Questo indica che il sistema ha raggiunto il suo limite teorico di *throughput* per questo specifico algoritmo.

L'incremento prestazionale osservabile tra 24, 32 e 48 *thread* è praticamente trascurabile (nell'ordine dello 1-2%). Avendo solo 6 *core* fisici, l'assegnazione di 8 *thread* per ogni core fisico genera un *overhead* di gestione dello *scheduler* del sistema operativo che annulla quasi interamente i benefici del parallelismo.

Il fatto che lo *speedup* non riesca a superare la barriera del 5.0x, nonostante i 6 *core* fisici disponibili, conferma che l'algoritmo *K-Means* su grandi *dataset* è limitato dalla larghezza di banda della memoria (*Memory Bandwidth Bound*). I *core* passano la maggior parte del tempo in attesa dei dati dalla RAM, rendendo inutile l'aggiunta di ulteriori *thread* logici.

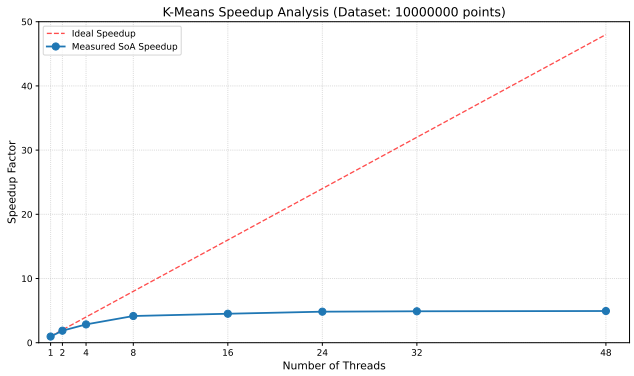

Figura 1: Speedup dell'implementazione SoA fino a 48 thread.

Il grafico dell'analisi dello *speedup* per il *dataset* da 10^7 punti sintetizza visivamente i limiti fisici dell'architettura *hardware* discussi precedentemente. In particolare di osserva:

- La linea tratteggiata rappresenta lo *speedup* ideale lineare. Si nota chiaramente come la curva misurata (*SoA Speedup*) inizi a divergere significativamente già dopo i 4 *thread*.
- Gomito della Curva (Saturazione *Hardware*): Il cambio di pendenza più marcato avviene tra gli 8 e i 16 *thread*. Questo riflette il passaggio dall'utilizzo dei *core* fisici puri all'utilizzo dell'*Hyper-Threading*. Una volta superati i 6 *core* fisici, il guadagno prestazionale rallenta drasticamente.
- Fase di Plateau: Dai 16 ai 48 *thread*, la curva diventa quasi una linea orizzontale. Questo *plateau* indica che aggiungere ulteriori *thread* logici (fino a 8 per ogni *core* fisico) non produce alcun beneficio reale, poiché le unità di calcolo e la larghezza di banda della memoria sono già completamente saturate.

In sintesi, il grafico è la prova visiva che la CPU ha raggiunto il punto di massima efficienza molto presto, e che l'*over-provisioning* dei *thread* oltre i 16 non è una strategia efficace.

7 Conclusioni

Si può concludere che per l'architettura in esame, il "*sweet spot*" prestazionale si colloca tra gli 8 e i 16 *thread*. Superare questa soglia aumenta solo la complessità di gestione senza produrre benefici tangibili, confermando che la scalabilità è dettata più dai limiti del *bus* di memoria e dei *core* fisici che dall'implementazione *software*.

8 Fonti

È possibile consultare il codice completo di questa implementazione nel seguente *repository GitHub* :

https://github.com/Elia29